08 · KI-Beschleuniger & Edge AI

Mikrocomputertechnik 3 · Von der Memory-Wall neuronaler Netze zu Systolic Arrays und TensorFlow Lite

Prof. Dr.-Ing. Daniel Klünder

Folie 1 / 73

Willkommen zu Block 8!

- Willkommen zur achten Vorlesung in Mikrocomputertechnik 3
- Block 7: OoO und Branch Prediction maximieren die General-Purpose-CPU
- Heute: Workload Künstliche Intelligenz — und warum Spezialhardware (NPUs) hilft
- Ziel: Memory Wall bei neuronalen Netzen verstehen und den Weg zu DSAs skizzieren



Notizen

Willkommen zu Block 8 knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Heute stehen regelmäßige Rechnung und Datenbewegung im Mittelpunkt. Eine NPU ist nicht für jede lokale Inferenz nötig. CPU, DSP und Vektoren können kleine Modelle tragen. Willkommen zur achten Vorlesung in Mikrocomputertechnik 3. Block 7: OoO und Branch Prediction maximieren die General-Purpose-CPU. Verständnischeck: Braucht jede lokale Inferenz eine NPU? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 2 / 73

Anknüpfung: Compiler-Labor

Kurz-Recap aus Block 7 (Vertiefung im Übungsblatt):

- Godbolt zeigt die Übersetzung, kein OoO-Timing
- `-O3` garantiert weder Unrolling noch kürzere Laufzeit
- Scheduling und Unrolling sind mögliche, keine sicheren Folgen
- Eine MAC-Kette bleibt RAW. Mehrere Akkumulatoren können unabhängig sein

```
/* Konzept: Unrolling von for (i = 0; i < 4; i++) a[i] = 0; */
a[0] = 0;
a[1] = 0;
a[2] = 0;
a[3] = 0;
```

Notizen

Anknüpfung: Compiler-Labor knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Vier Arrayzeilen sind manuelles Unrolling, kein Compileroutput. Eine Summe sum plus gleich w mal x behält eine RAW-Kette. Godbolt zeigt Übersetzung, kein OoO-Timing. -O3 garantiert weder Unrolling noch kürzere Laufzeit. Godbolt zeigt die Übersetzung, kein OoO-Timing. `-O3` garantiert weder Unrolling noch kürzere Laufzeit. Verständnischeck: Beweist die Vier-Zeilen-Folge einen -O3-Output? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 3 / 73

Grenze der General-Purpose-CPU

- Mehr Issue-Breite verteuert Scheduler, Ports und Bypass. Quadratisch gilt nicht pauschal für ROB, Renaming und Decode zusammen
- OoO kann Loads überlappen. Es hebt eine einzelne Akkumulatorkette nicht auf
- DLP ergänzt ILP. SIMD ist eine Umsetzung, keine Definition von DLP

Notizen

Grenze der General-Purpose-CPU knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. OoO kann Loads und unabhängige Produkte überlappen. Eine einzelne Akkumulatorkette bleibt abhängig. DLP ist gleiche Rechnung an vielen Werten. SIMD ist eine Umsetzung davon, nicht die Definition. Mehr Issue-Breite verteuert Scheduler, Ports und Bypass. Quadratisch gilt nicht pauschal für ROB, Renaming und Decode zusammen. OoO kann Loads überlappen. Es hebt eine einzelne Akkumulatorkette nicht auf. Verständnischeck: Kann OoO bei einem Dot Product helfen? Ja, aber die Summenkette bleibt. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde

nicht kompiliert, trainiert oder ausgeführt.

Folie 4 / 73

Neuer Workload: Edge AI

- Edge AI umfasst Endgerät und Gateway. Nicht jede lokale Inferenz braucht eine NPU
- Beispiele: Schrittzählung, Wake-Word, Vibrationsdiagnose an Maschinen
- Dauerbetrieb auf knappen Ressourcen (Energie, SRAM, Flash)

Notizen

Neuer Workload: Edge AI knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Edge umfasst Endgerät und Gateway. Training bestimmt Parameter, Inferenz verwendet sie. Eine Aktivitätsklasse ist keine Schrittzahl. Eine Schwellenwertlösung kann reichen. Edge AI umfasst Endgerät und Gateway. Nicht jede lokale Inferenz braucht eine NPU. Beispiele: Schrittzählung, Wake-Word, Vibrationsdiagnose an Maschinen. Verständnischeck: Ist ein lokales Gateway Edge? Ja. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 5 / 73

Cloud AI vs. Edge AI

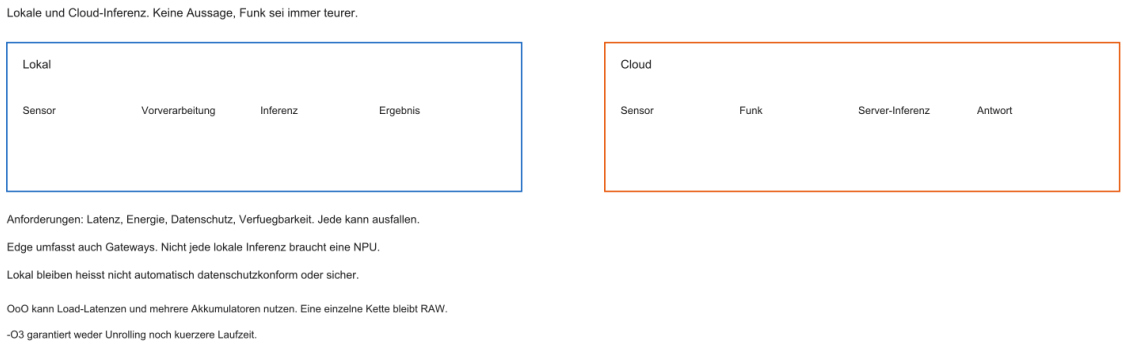
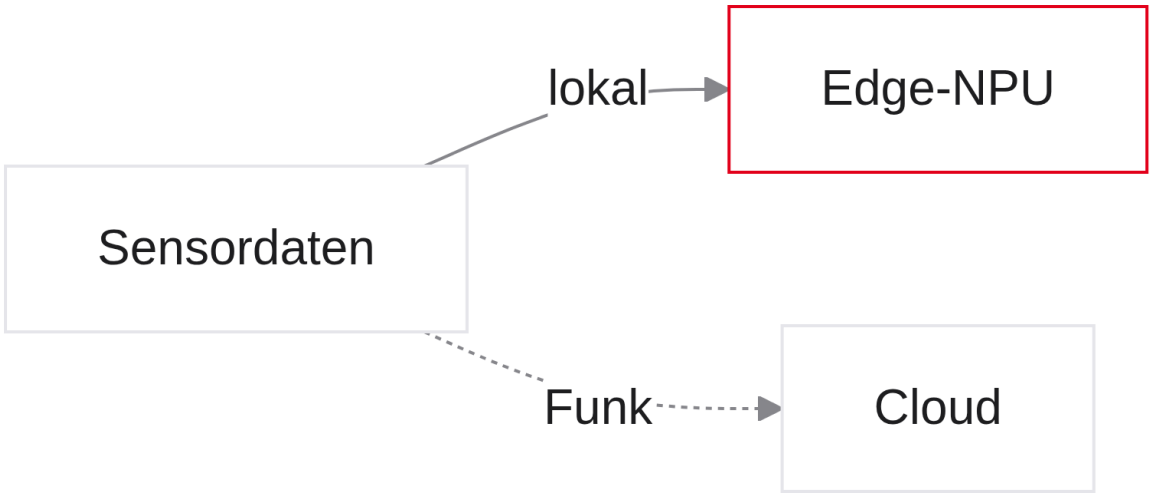


Abbildung 1: Lokal und Cloud sind Optionen. Funk ist nicht immer die teurere Seite.

Warum nicht alles in die Cloud?

1. Latenz — sicherheitskritische Reaktionen dulden keine Funkwartezeit
2. Privacy — Audio/Video sollen oft lokal bleiben
3. Energie und Verfügbarkeit sind Anforderungen. Funk ist nicht immer teurer als lokale Rechnung



Notizen

Cloud AI vs. Edge AI knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Lokal kann CPU, DSP, NPU oder Gateway rechnen. Lokalität ist keine Datenschutzgarantie. Kommunikation ist nicht stets teurer als Rechnen. Verständnischeck: Kann lokale Rechnung mehr Energie brauchen als Übertragung? Ja. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 6 / 73

Agenda

1. KI-Herausforderung und Memory Wall
2. DSPs und SIMD
3. RISC-V Vector Extension (RVV)
4. NPU und Systolic Arrays
5. Edge-AI-Software / TensorFlow Lite

Notizen

Agenda knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Die Agenda fragt nach Bytes, Parallelität, Vektorlänge, lokaler Wiederverwendung und Firmware. Das sind kombinierbare Wege, keine Pflichttreppe. TFLM ist hier nicht als Labor vorhanden. Verständnischeck: Warum Speicher vor dem Array? Weil lokale Wege Verkehr vermeiden sollen. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 7 / 73

Lernziele Block 8

- Memory Wall bei Matrix-/MAC-Lasten erklären
- MAC-Operation und ihre Bedeutung benennen
- ILP vs. DLP unterscheiden
- Systolic Arrays einordnen
- TensorFlow Lite Micro Workflow skizzieren

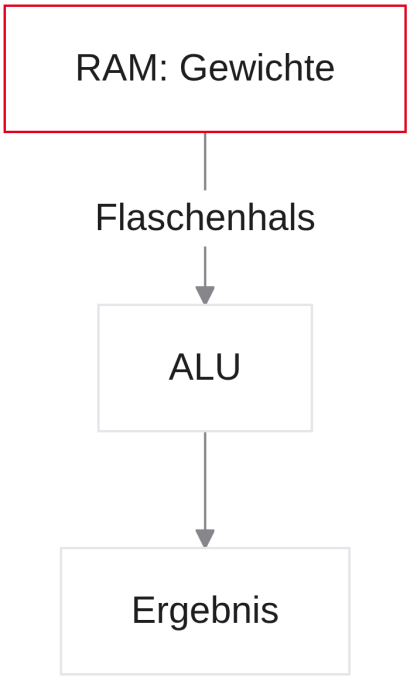
Notizen

Lernziele Block 8 knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Prüfbar sind die Dense-Ausgabe, Parameter gegen MACs, die Speichergrenze, SIMD gegen DLP, das zurückgegebene VL und die TFLM-Lebensdauer. TOPS brauchen Datentyp und Zählkonvention. Memory Wall bei Matrix-/MAC-Lasten erklären. MAC-Operation und ihre Bedeutung benennen. Verständnischeck: Was zuerst zu TOPS? Datentyp, Ops je MAC und Peak gegen Anwendung. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 8 / 73

Die Leitfrage des Tages

- Kleines Netz: z. B. 10^6 Gewichte — Inferenz verrechnet sie mit Aktivierungen
- SRAM, Cache-Hit, Flash und DRAM haben verschiedene Kosten. Nicht jeder Zugriff ist ein DRAM-Miss
- Wie rechnet man Millionen MACs/s auf knapper Energie, ohne am Speicher zu ersticken?



Notizen

Die Leitfrage des Tages knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Eine Million Float32-Gewichte sind 4 MB, als Int8 1 MB. Bias und Aktivierungen fehlen. Nicht jede MCU hat DRAM. Eine Million Gewichte sind nur bei einmaliger Nutzung eine Million MACs. Kleines Netz: z. B. 10^6 Gewichte — Inferenz verrechnet sie mit Aktivierungen. SRAM, Cache-Hit, Flash und DRAM haben verschiedene Kosten. Nicht jeder Zugriff ist ein DRAM-Miss. Verständnischeck: Sind eine Million Gewichte eine Million MACs? Nur in diesem Sonderfall. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 9 / 73

Aufgabe — Sechs MACs

$$W = \begin{bmatrix} 1 & 2 & 3 \\ 0 & -1 & 4 \end{bmatrix}, x = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}, b = \begin{bmatrix} 1 \\ -2 \end{bmatrix}.$$

Warum sind sechs Gewichte hier sechs produktbezogene MACs, bei einer Faltung aber nicht? Was ist y?

Notizen

Aufgabe — Sechs MACs knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. W ist [[1,2,3],[0,-1,4]], x ist [2,1,3], b ist [1,-2]. Die erste Zeile ist 1 mal 2 plus 2 mal 1 plus 3 mal 3 plus 1 gleich 14. Die zweite ist 0 plus minus 1 plus 12 minus 2 gleich 9. Sechs Gewichte und zwei Bias sind acht Parameter und sechs produktbezogene MACs. Verständnischeck: Wie viele Parameter? Acht. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 10 / 73

Neuronale Netze auf Hardware-Ebene

- Abseits des Hypes: große Zahlenarrays
- Inputs / Activations: Sensordaten (z. B. 28×28 Pixel)
- Weights: trainierte Konstanten (oft Flash)
- Neuron: gewichtete Summe der Eingänge (+ Aktivierung, hier ausgeblendet)

Notizen

Neuronale Netze auf Hardware-Ebene knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Aktivierungen sind auch Zwischenergebnisse. 28 mal 28 ist 784 nur bei einem Kanal. Gewichte können im Flash liegen, Summen oft in SRAM oder Registern. Eine Matrix ist nicht das ganze Netz. Abseits des Hypes: große Zahlenarrays. Inputs / Activations: Sensordaten (z. B. 28×28 Pixel). Verständnischeck: Sind alle Aktivierungen Sensorwerte? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 11 / 73

Mathematik: Matrix / Dot Product

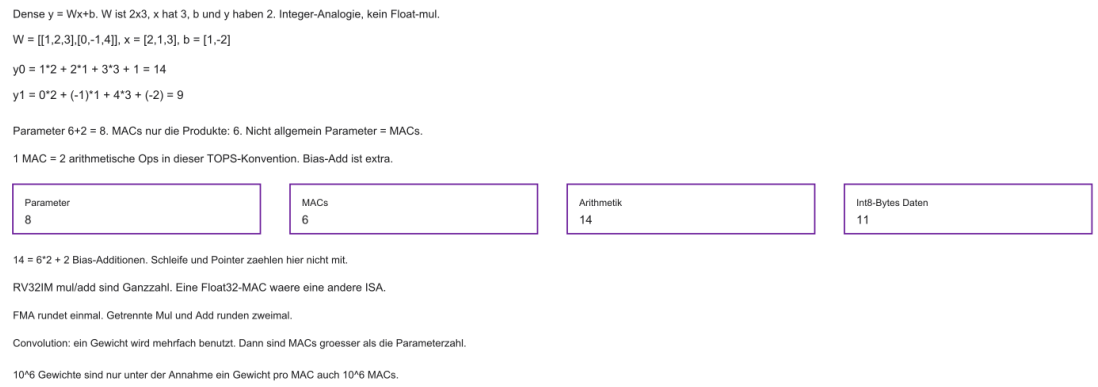


Abbildung 2: y0 = 14 und y1 = 9. Sechs MACs, acht Parameter, 14 arithmetische Operationen.

```
float sum = 0.0f;
for (int i = 0; i < N; i++) {
    sum += input[i] * weights[i];
}
```

- Kein Kontrollfluss-Chaos: vorhersagbarer Datenstrom
- Naive Lehrzählung ohne Registerreuse: zwei Loads, eine Multiplikation, eine Addition. Das ist keine Übersetzung in Assembler

Notizen

Mathematik: Matrix / Dot Product knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Sechs MACs sind in der benannten Konvention zwölf arithmetische Ops plus zwei Biasadditionen, also vierzehn. Das sind keine Takte. Int8-Gewichte, Int32-Bias und Int8-Eingaben sind 6 plus 8 plus 3 gleich 17 Byte, optional plus zwei Ausgaben 19. Elf Byte wäre ein anderes, unvollständiges Budget. Kein Kontrollfluss-Chaos: vorhersagbarer Datenstrom. Pro Iteration: zwei Loads + Mul + Add. Verständnischeck: Warum nicht acht MACs? Die Biaswerte werden separat addiert. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 12 / 73

MAC-Operation

- Multiply-Accumulate: $a \leftarrow a + (w \times x)$
- w: Gewicht, x: Activation, a: Akkumulator
- KI-Chips: oft in TOPS (viele MAC-Äquivalente / s) spezifiziert — nicht nur in MHz

Notizen

MAC-Operation knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. a gleich 5, w gleich 2, x gleich 3 ergibt 11. Eine MAC kann eine Instruktion und trotzdem zwei Ops sein. Float-FMA rundet einmal, getrennte Mul und Add zweimal. Eine MAC ist nicht automatisch ein Takt. Multiply-Accumulate: $a \leftarrow a + (w \times x)$. w: Gewicht, x: Activation, a: Akkumulator. Verständnischeck: Ist eine MAC immer ein Takt? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 13 / 73

Anatomie einer MAC auf RISC-V

Vereinfachte Befehlsfolge:

```
# Konzept – RV32IM (mul); Adressen schematisch
lw  t0, 0(s1)     # x
lw  t1, 0(s2)     # w
mul t2, t0, t1
add t3, t3, t2    # a += w*x
```

- `mul`/`add` sind hier Ganzzahl, die C-Schleife darüber ist Float. Das ist eine Analogie, keine Float-ISA
- Zwei Loads und zwei Ops gelten nur ohne Wiederverwendung und ohne FMA

Notizen

Anatomie einer MAC auf RISC-V knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. lw lädt ein 32-Bit-Muster. mul und add sind hier Integer der M-Erweiterung, keine Float-Rechnung. Zwei Loads sind nicht zwei DRAM-Misses. Der Akkumulator muss vorher gesetzt sein. Lehrcode, nicht assembliert. `mul`/`add` sind hier Ganzzahl, die C-Schleife darüber ist Float. Das ist eine Analogie, keine Float-ISA. Zwei Loads und zwei Ops gelten nur ohne Wiederverwendung und ohne FMA. Verständnischeck: Warum reicht RV32I nicht für mul? Es braucht die M-Erweiterung. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Arithmetic Intensity: Zahlenbeispiel

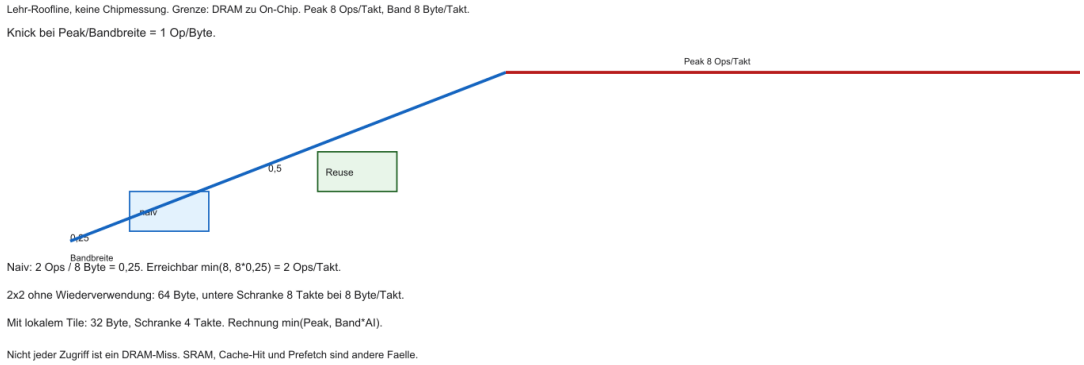


Abbildung 3: Knick bei 1 Op/Byte. Naiv 0,25, mit Wiederverwendung 0,5. Lehrwerte, keine Messung.

- AI $=$ Ops$/$Bytes
- Grenze dieses Beispiels: DRAM zum On-Chip-Speicher. Output und Bias zählen hier noch nicht
- 2 Ops und 8 Byte ergeben $0{,}25$ Ops/Byte. Das ist nicht allgemein „unter 1 = memory-bound“
- Erreichbar ist min$(P_{peak}, Bandbreite \times AI)$. Der Knick liegt bei $P_{peak}/Bandbreite$

Notizen

Arithmetic Intensity: Zahlenbeispiel knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. I_arith ist Ops je Byte über die Grenze DRAM nach On-Chip. Peak 8 Ops je Takt, Bandbreite 8 Byte je Takt, Knick bei 1 Op je Byte. Bei 0,25 sind höchstens 2 Ops je Takt erreichbar, bei 0,5 höchstens 4. Das ist eine Obergrenze, keine Messung. AI $=$ Ops$/$Bytes. Grenze dieses Beispiels: DRAM zum On-Chip-Speicher. Output und Bias zählen hier noch nicht. Verständnischeck: Warum liegt der Knick bei 1? 8 Ops geteilt durch 8 Byte. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 15 / 73

Datenproblem: Gewichte

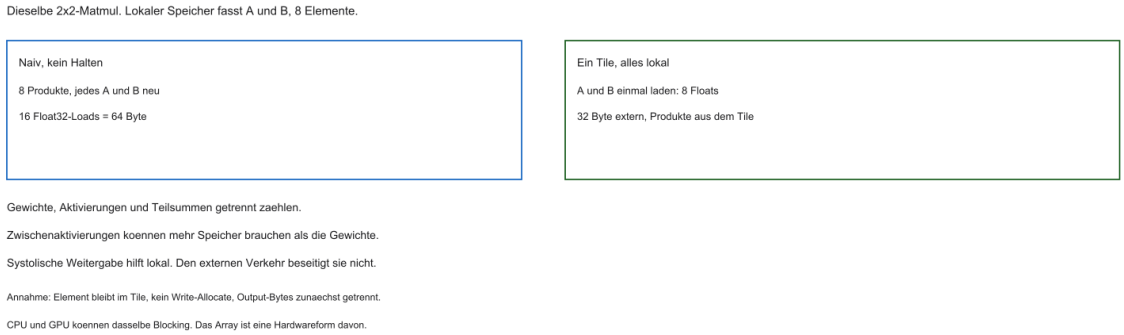


Abbildung 4: Dieselbe 2×2-Rechnung: 64 Byte naiv, 32 Byte aus einem Tile.

- Modelle wie ResNet-50: Millionen Parameter — Megabyte-Bereich
- L1 oft nur Dutzende KiB → Matrizen passen nicht
- Ein Streaming-Datensatz erzeugt nicht bei jedem Skalarzugriff einen Capacity Miss. Cachezeilen und Prefetch bündeln Zugriffe

Notizen

Datenproblem: Gewichte knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. A ist [[1,2],[3,4]], B ist [[5,6],[7,8]], C ist [[19,22],[43,50]]. Acht MACs sind 16 Ops. Ohne Halten 16 mal 4 Byte gleich 64 Byte, mit Tile 32 Byte. Outputs fehlen in dieser Zahl. Auch eine CPU kann Tiles bilden. Modelle wie ResNet-50: Millionen Parameter — Megabyte-Bereich. L1 oft nur Dutzende KiB → Matrizen passen nicht. Verständnischeck: Warum halbieren sich die Eingangsbytes? Jeder Wert wird einmal geladen und lokal wiederverwendet. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 16 / 73

Memory Wall bei KI

Phase	Typische Dauer (schematisch)
Load x, w (DRAM)	viele Takte
MUL + ADD	wenige Takte
nächster Load	wieder viele Takte

- ALU oft idle; Energie geht in Wartezeit und Datentransport

Notizen

Memory Wall bei KI knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Die Tabelle ist ein pessimistischer serieller Pfad, nicht jede echte Latenz. Requests können sich überlappen. Vier Transfers mit Latenz 3: seriell bis Takt 12, gepipelinet bis Takt 6, wenn ein Start je Takt erlaubt ist. ALU oft idle; Energie geht in Wartezeit und Datentransport. Verständnischeck: Warum nicht jede Miss-Latenz addieren? Zugriffe können sich überlappen. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 17 / 73

Rechenbeispiel: 10^6 MACs

- 10^6 Gewichte sind nur dann 10^6 MACs, wenn jedes Gewicht genau einmal multipliziert wird
- Eine Faltung nutzt Gewichte mehrfach. Sequenz, Bild und Batch ändern den Aufwand
- Einzelne Miss-Latenzen darf man nicht zu einer Summe addieren, wenn Zugriffe überlappen

Notizen

Rechenbeispiel: 10^6 MACs knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Eine Million MACs, einmal je Gewicht, sind zwei Millionen Ops. Naiv zwei Float32-Operanden sind acht Millionen Byte. Bei 8 Ops und 8 Byte je Takt sind das 250000 Rechen- und eine Million Transfertakte. Kein Boardwert. 10^6 Gewichte sind nur dann 10^6 MACs, wenn jedes Gewicht genau einmal multipliziert wird. Eine Faltung nutzt Gewichte mehrfach. Sequenz, Bild und Batch ändern den Aufwand. Verständnischeck: Welche Untergrenze dominiert? Der Transfer. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 18 / 73

Warum OoO hier wenig hilft

- OoO/Predictor: stark bei unregelmäßigem Kontrollfluss
- MAC-Schleife: linear, hoch vorhersagbar
- Der Predictor ist bei einer festen Schleife nicht wertlos, aber selten der Engpass
- Unabhängige Akkumulatoren kann OoO ausführen. Die Speichergrenze bleibt, wenn die Arithmetic Intensity niedrig ist

Notizen

Warum OoO hier wenig hilft knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. OoO hilft bei Überlappung, hebt aber die Bandbreitengrenze nicht. Eine Summe bleibt RAW-abhängig. Niedrige Intensity beweist nicht, dass der Scheduler nutzlos ist. OoO/Predictor: stark bei unregelmäßigem Kontrollfluss. MAC-Schleife: linear, hoch vorhersagbar. Verständnischeck: Beweist niedrige Intensity

Nutzlosigkeit? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 19 / 73

Fazit: CPU vs. KI-Last

- ALU-Auslastung bei naiver Matrixrechnung oft gering
- Viel Zeit/Energie in Transport und Instruktionsverwaltung
- Spezialhardware kann passen, wenn Operatoren, Form und Auslastung sie nutzen

Notizen

Fazit: CPU vs. KI-Last knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Geringe Auslastung folgt hier aus Versorgung oder Abhängigkeit, nicht aus jeder KI auf der CPU. Tiling kann helfen, bevor neue Hardware kommt. ALU-Auslastung bei naiver Matrixrechnung oft gering. Viel Zeit/Energie in Transport und Instruktionsverwaltung. Verständnischeck: Welche Softwaretechnik zuerst? Tiling innerhalb der Speichergrenze. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 20 / 73

Lösung: Data-Level Parallelism (DLP)

- Weg von „viele verschiedene Instruktionen“ (ILP)
- DLP: dieselbe Rechnung an vielen Werten. SIMD ist eine mögliche Umsetzung, nicht die Definition
- Hundert unabhängige Produkte brauchen weder hundert Rechenwerke noch einen Takt

Notizen

Lösung: Data-Level Parallelism (DLP) knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Hundert unabhängige Produkte sind DLP. Sie brauchen weder hundert Units noch einen Takt. Ein Befehl für viele Daten ist SIMD, nicht die Definition von DLP. Weg von „viele verschiedene Instruktionen“ (ILP). DLP: dieselbe Rechnung an vielen Werten. SIMD ist eine mögliche Umsetzung, nicht die Definition. Verständnischeck: Garantiert ein Befehl hundert Produkte im selben Takt? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 21 / 73

Domain-Specific Architecture (DSA)

- Eine DSA kann Formate und Operatoren einschränken. Programmierbarkeit muss nicht vollständig entfallen
- DSA: Silizium für eine Domäne (z. B. Matrizen) — oft deutlich effizienter
- Evolutionsschritte: SIMD → DSP-Tricks → Vektoren → NPUs

Notizen

Domain-Specific Architecture (DSA) knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Eine DSA passt Silizium an ein Muster an und kann Operatoren einschränken. SIMD, DSP, RVV und NPU können im SoC zusammenarbeiten. Kein Ersatzpfad ist vorgeschrieben. Extreme DLP-Optimierung → Verlust der Allzweck-Fähigkeit. DSA: Silizium für eine Domäne (z. B. Matrizen) — oft deutlich effizienter. Verständnischeck: Muss der DSP durch eine NPU ersetzt werden? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 22 / 73

Fragen? — MAC und Memory Wall

- Was leistet die MAC-Operation im Netz?
- Warum kehrt die Memory Wall bei KI zurück?
- ILP vs. DLP?

Notizen

Fragen? — MAC und Memory Wall knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. MAC verbindet Produkt und Summe. ILP und DLP können zugleich vorliegen. Die Memory Wall ist hier Transport, die RAW-Kette ist Abhängigkeit. Was leistet die MAC-Operation im Netz?. Warum kehrt die Memory Wall bei KI zurück?. Verständnischeck: Kann ein Programm beides haben? Ja. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 23 / 73

Was bedeutet SIMD?

- SIMD: Single Instruction, Multiple Data
- Paradebeispiel für DLP
- Ein Fetch/Decode → intern mehrere parallele Operationen

Notizen

Was bedeutet SIMD knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Ein Decode steuert gleiche Operationen auf getrennten Lanes. Vier Lanes sind nicht vier Befehle. Weniger Fetch garantiert keine Gesamtenergieeinsparung. SIMD: Single Instruction, Multiple Data. Paradebeispiel für DLP. Verständnischeck: Sind vier Lanes vier Befehle? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

SIMD-Prinzip

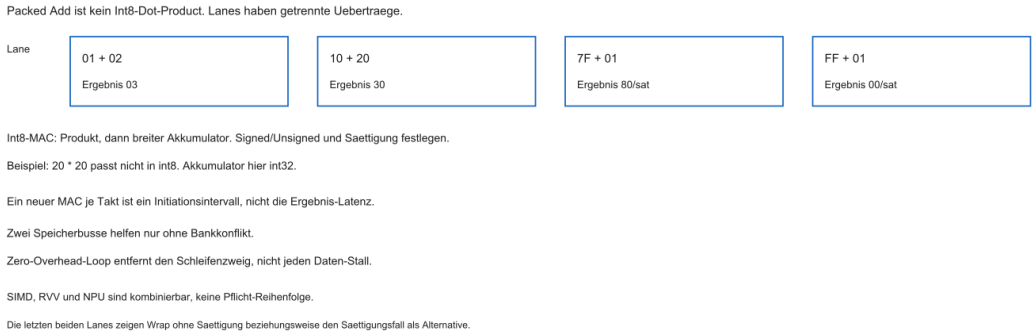
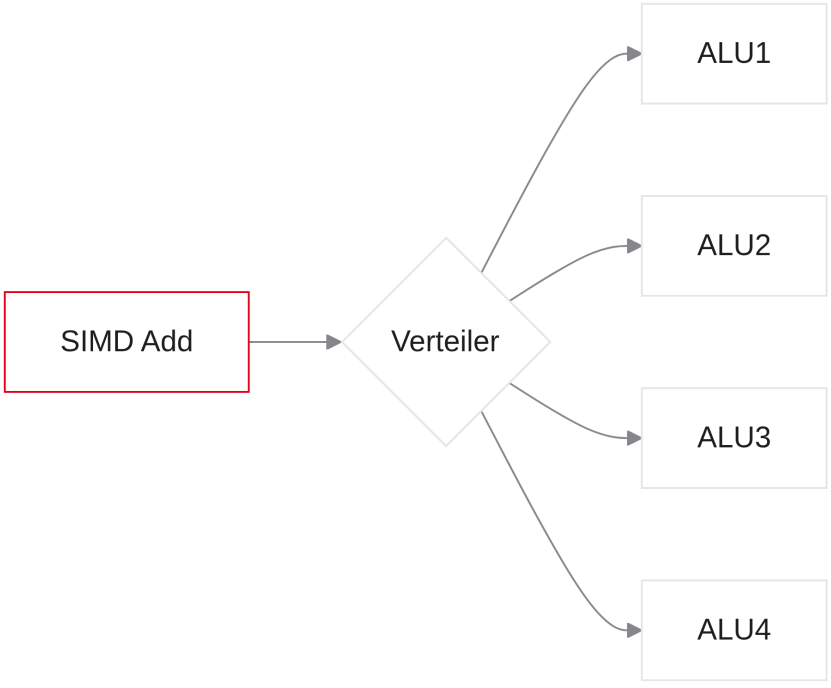


Abbildung 5: Vier 8-Bit-Lanes addieren getrennt. Ein Int8-Produkt braucht einen breiteren Akkumulator.

- Skalar (SISD): `add` → eine Addition
- SIMD: ein Befehl → mehrere parallele Adds auf Daten-Lanes



Notizen

SIMD-Prinzip knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Unsigned modulo 256: 01 plus 02 ist 03, 10 plus 20 ist 30, 7F plus 01 ist 80, FF plus 01 ist 00. Das ist Addition, kein Dot Product. 20 mal 20 gleich 400 passt nicht in ein Byte. Signed und Sättigung sind andere Verträge. Skalar (SISD): `add` → eine Addition. SIMD: ein Befehl → mehrere parallele Adds auf Daten-Lanes. Verständnischeck: Warum kein Übertrag in die Nachbarlane? Jede Lane rechnet modulo 256 für sich. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 25 / 73

Packed SIMD

- Problem: schmale ISA-Register, breite Parallelität?
- Lösung: Register in Lanes zerlegen (z. B. 32 Bit → 4×8 Bit)
- SIMD-Op: Lane für Lane parallel, ohne Carry zwischen Lanes

Notizen

Packed SIMD knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Ein normales 32-Bit-add darf Bytes überlaufen. Packed SIMD braucht eine Instruktion mit Lanevertrag. Vier gepackte Bytes und ein gewöhnliches add genügen nicht. Problem: schmale ISA-Register, breite Parallelität?. Lösung: Register in Lanes zerlegen (z. B. 32 Bit → 4×8 Bit). Verständnischeck: Genügt normales add? Nein, Überträge vermischen die Felder. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 26 / 73

Digital Signal Processors (DSPs)

- Vor dem KI-Boom: DSPs für Audio, Modem, Filter
- Algorithmen (FIR etc.): wie KI stark Array-/Skalarprodukt-lastig
- Vorläufer vieler NPU-Ideen

Notizen

Digital Signal Processors (DSPs) knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Ein FIR ist y von n gleich Summe h von k mal x von n minus k. Mit h gleich 1 und 2 und Fenster 3 und 4 ist das 11. Dense ähnelt der Summe, der zeitliche Kontext kann anders sein. Vor dem KI-Boom: DSPs für Audio, Modem, Filter. Algorithmen (FIR etc.): wie KI stark Array-/Skalarprodukt-lastig. Verständnischeck: Was teilen FIR und Dense? Viele Produkte in einer Summe. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 27 / 73

DSP: dedizierte MAC-Einheit

- Mul und Add oft fest verdrahtet als eine MAC
- $a + (x \times w)$ in wenigen Takten / oft einem Zyklus (je nach Design)
- Weniger Instruktionsfolge als auf skalarer RISC-ALU

Notizen

DSP: dedizierte MAC-Einheit knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Initiierungsintervall 1 erlaubt einen neuen unabhängigen Start je Takt. Die Ergebnislatenz kann trotzdem mehrere Takte sein. Eine abhängige Summe braucht Weiterleitung oder Warten. Mul und Add oft fest verdrahtet als eine MAC. $a + (x \times w)$ in wenigen Takten / oft einem Zyklus (je nach Design). Verständnischeck: Beweist ein Start je Takt Latenz 1? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 28 / 73

Super-Harvard in DSPs

- Mehrere parallele Speicherpfade gegen die Memory Wall
- Typisch: Code-Bus + getrennte Datenbusse für x und w
- Loads und MAC können im selben Takt überlappen

Notizen

Super-Harvard in DSPs knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Super-Harvard ist eine benannte DSP-Idee, nicht jeder DSP. Zwei Busse garantieren nicht zwei Operanden, wenn dieselbe Bank kollidiert. Mehrere parallele Speicherpfade gegen die Memory Wall. Typisch: Code-Bus + getrennte Datenbusse für x und w. Verständnischeck: Garantieren zwei Busse zwei Operanden? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 29 / 73

Zero-Overhead Loops

- Hardware-Schleifenzähler: Start, Ende, Count
- Kein `beq` pro Iteration — kein Predictor-Overhead
- PC-Wrap in Hardware bis Count abgelaufen

Notizen

Zero-Overhead Loops knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Ein Hardwareloop spart bestimmte Branch- und Zählerbefehle. Er entfernt keine Speicherwartezeit. Die spätere RVV-Schleife mit bnez ist Software. Hardware-Schleifenzähler: Start, Ende, Count. Kein `beq` pro Iteration — kein Predictor-Overhead. Verständnischeck: Entfernt der Hardwareloop Speicherwarten? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 30 / 73

Grenze: Packed SIMD skaliert schlecht

- Feste Vektorbreite in der ISA (z. B. 128 / 256 / 512 Bit)
- Alter Code nutzt neue breitere Units nicht automatisch
- Oft Neukompilieren / Intrinsics für jede Generation

Notizen

Grenze: Packed SIMD skaliert schlecht knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. 128 Bit fassen vier Int32-Werte. Alter Code bleibt korrekt, nutzt breitere Befehle aber nicht automatisch. Neukompilieren ist eine Möglichkeit, keine Pflicht für Korrektheit. Feste Vektorbreite in der ISA (z. B. 128 / 256 / 512 Bit). Alter Code nutzt neue breitere Units nicht automatisch. Verständnischeck: Wie viele Int32 in 128 Bit? Vier, ohne Taktaussage. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 31 / 73

Instruction-Fetch-Overhead bleibt

- Auch breites SIMD: weiterhin viele Instruktionen für große Matrizen
- Decode und I-Cache kosten Energie
- Wunsch: „rechne lange auf diesen Daten“ mit weniger Instruction Stream

Notizen

Instruction-Fetch-Overhead bleibt knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. SIMD senkt Befehle je Element, nicht den Fetch auf null. Loads und Schleifen bleiben. Weniger Fetch ist kein Joule-Nachweis. Auch breites SIMD: weiterhin viele Instruktionen für große Matrizen. Decode und I-Cache kosten Energie. Verständnischeck: Verschwindet Fetch? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 32 / 73

Übergang zu Vektoren

- Packed SIMD hat eine feste Breite. RVV wählt die aktive Elementzahl zur Laufzeit
- RISC-V Vector Extension (RVV): Ansatz für skalierbare Vektor-ISAs
- Code-Idee: dieselbe Software auf schmalen und breiten Implementierungen

Notizen

Übergang zu Vektoren knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. RVV wählt die aktive Länge zur Laufzeit. Ein Register bleibt endlich. Ein langes Array wird in Stücken verarbeitet. Dieselbe Binärdatei braucht passende ISA und ABI. Weg von starrem Packed SIMD → flexible Vektorlängen. RISC-V Vector Extension (RVV): Ansatz für skalierbare Vektor-ISAs. Verständnischeck: Kann ein Register das ganze Array halten? Nicht beliebig lang. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 33 / 73

Kurzfazit: DSP und SIMD

- DLP ist die Datenparallelität. SIMD nutzt sie mit einem gemeinsamen Operationstyp
- Packed SIMD: breite Register, starre Breiten
- DSPs: MAC, Multi-Bus, Zero-Overhead Loops
- Vektoren und NPUs können bei passenden Operatoren helfen. Ein DSP ist dadurch nicht veraltet

Notizen

Kurzfazit: DSP und SIMD knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. DLP ist nicht der Name von SIMD. Packed SIMD hat feste Breite. DSP-MAC, Busse und Hardwareloops sind getrennte Merkmale. Vektoren sind kein Beweis, dass DSPs veraltet sind. DLP ist die Datenparallelität. SIMD nutzt sie mit einem gemeinsamen Operationstyp. Packed SIMD: breite Register, starre Breiten. Verständnischeck: Ist DLP nur ein anderer Name für SIMD? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 34 / 73

Fragen? — SIMD und DSP

- Wie senkt SIMD den Fetch/Decode-Overhead?
- Warum ist eine Hardware-MAC für Signalverarbeitung wertvoll?
- Warum muss Packed-SIMD-Code oft neu angepasst werden?

Notizen

Fragen? — SIMD und DSP knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. SIMD senkt Fetch je Lanegruppe. Die Hardware-MAC passt zur Produktsumme, wenn die Daten da sind. Feste Breite wird nicht automatisch größer, nur weil neue Hardware breiter ist. Wie senkt SIMD den Fetch/Decode-Overhead?. Warum ist eine Hardware-MAC für Signalverarbeitung wertvoll?. Verständnischeck: Warum ist die MAC wertvoll? Sie bündelt die häufige Rechnung, wenn die Versorgung passt. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 35 / 73

Echte Vektor-Architektur

- Verarbeitet lineare Arrays (Vektoren), nicht nur Skalare
- Anders als Packed SIMD: Vektorlänge flexibel, nicht in feste Registerhäppchen geschnitten
- Befehlssatz abstrahiert: „Addiere Vektor A und B“ — unabhängig von der Implementierungsbreite

Notizen

Echte Vektor-Architektur knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Ein Vektorbefehl hat eine konfigurierte Länge. Ist N größer als VLMAX, folgen mehrere Stücke. VLEN ist nicht die Lanezahl. Ein Load ist keine Garantie, dass alles in einem Takt fertig ist. Verarbeitet lineare Arrays (Vektoren), nicht nur Skalare. Anders als Packed SIMD: Vektorlänge flexibel, nicht in feste Registerhäppchen geschnitten. Verständnischeck: Was passiert bei zu langem Array? Mehrere VL-Stücke. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 36 / 73

Vektor-Register-File

- Neben `x0`–`x31`: Vektorregister `v0`–`v31`
- Jedes `v`-Register hält viele Elemente gleichzeitig
- Kapazität hängt von der Implementierung ab (VLEN = Vector Length)

Notizen

Vektor-Register-File knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Es gibt 32 Vektorregister zu je VLEN Bit. Bei VLEN 128, SEW 32 und LMUL 1 ist VLMAX 4, nicht 128. VL ist die aktive Elementzahl. Eine Lanegruppe kann diese Elemente über mehrere Takte rechnen. Neben `x0`–`x31`: Vektorregister `v0`–`v31`. Jedes `v`-Register hält viele Elemente gleichzeitig. Verständnischeck: Ist VLEN 128 gleich 128 Elemente? Nein, bei 32 Bit sind es vier. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 37 / 73

RISC-V Vector Extension (RVV)

- Offizielle RISC-V-Erweiterung für Vektormathematik (u. a. ML, Krypto)
- Neue Instruktionen, z. B. `vadd.vv`
- Software kennt die physische Breite oft nicht (Watch vs. Server)
- Frage: ein Binary für unbekannte VLEN?

Notizen

RISC-V Vector Extension (RVV) knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. vadd.vv addiert Integer-Vektoren. Krypto und Int8-TinyML brauchen passende Erweiterungen. Dieselbe Binärdatei läuft nicht auf jeder Uhr und jedem Server. Offizielle RISC-V-Erweiterung für Vektormathematik (u. a. ML, Krypto). Neue Instruktionen, z. B. `vadd.vv`. Verständnischeck: Läuft dasselbe Binary überall nur wegen RVV? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 38 / 73

`vsetvli` / hardware-agnostisch

- Software: „Array mit N Elementen“
- Bei LMUL $= 1$ ist VLMAX $=$ VLEN$/$SEW. Allgemein VLMAX $=$ LMUL $\times$ VLEN$/$SEW
- `vsetvli` liefert nicht immer min$(AVL, VLMAX)$
- Schleife läuft in Hardware-Schritten durch das Array

```
# RVV 1.0, LMUL=1, e32, ta, ma. Nicht in diesem Repo assembliert
loop:
    vsetvli t0, a0, e32, m1, ta, ma
    vle32.v v1, (a1)
    vle32.v v2, (a2)
    vadd.vv v3, v1, v2
    vse32.v v3, (a3)
    slli t1, t0, 2
    add a1, a1, t1
    add a2, a2, t1
    add a3, a3, t1
    sub a0, a0, t0
    bnez a0, loop
```

Notizen

`vsetvli` / hardware-agnostisch knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. a0 ist der Rest, a1 und a2 sind Quellen, a3 das Ziel, t0 das zurückgegebene VL. slli mal 4 rechnet Bytes, nicht Datenwerte. Ist AVL höchstens VLMAX, ist VL gleich AVL. Liegt AVL zwischen VLMAX und dem Doppelten, liegt VL zwischen der Hälfte aufgerundet und VLMAX. Ab dem Doppelten ist VL gleich VLMAX. Bei N gleich 0 passiert nichts. ta und ma dürfen nicht als neue Ergebnisse gelesen werden. Lehrcode, nicht assembliert. Software: „Array mit N Elementen“. Bei LMUL $= 1$ ist VLMAX $=$ VLEN$/$SEW. Allgemein VLMAX $=$ LMUL $\times$ VLEN$/$SEW. Verständnischeck: Warum um VL vorrücken? Die Hardware darf im mittleren Bereich kürzer wählen. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 39 / 73

Ein Binary, viele VLEN

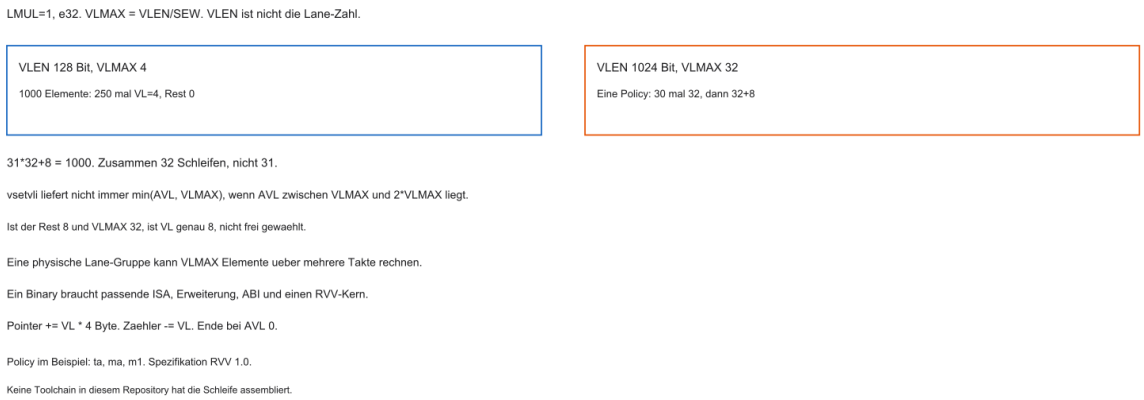


Abbildung 6: 128 Bit: 250 Schritte zu VL 4. 1024 Bit: 31 Schritte zu VL 32 plus VL 8.

Beispiel: 1000 Elemente, `vadd.vv` (schematisch)

Plattform	Breite (schematisch)	VL (e32)	Schleifenläufe
Edge	schmal (z. B. 128 Bit)	4	250
Server	1024 Bit, LMUL 1	32, dann 20+20 oder 32+8	32

Gleiche RVV-Schleifenidee skaliert mit VLEN — ohne feste Packed-SIMD-Breite im Quelltext.

Notizen

Ein Binary, viele VLEN knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. 1000 Elemente, e32, m1: VLEN 128 ergibt VLMAX 4 und 250 Iterationen. VLEN 1024 ergibt VLMAX 32. Nach 30 vollen Stücken bleiben 40. Die erste VL darf 20 bis 32 sein, der Rest ist dann exakt. 20 plus 20, 24 plus 16 und 32 plus 8 sind zulässige Policies mit 32 Iterationen. Je Zeiger insgesamt 4000 Byte. 31 mal 32 plus 8 ist nur eine Policy. Verständnischeck: Ist bei AVL 40 und VLMAX 32 zwingend VL 32? Nein, 20 bis 32 sind erlaubt. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist

Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 40 / 73

Energie: weniger Instruction Fetch

- Architektonisch viele Elemente je Befehl. Physisch kann dieselbe Lane sie über mehrere Takte rechnen
- Skalar: dutzende Loads/Adds/Branches aus dem I-Cache
- Weniger Fetch kann Energie sparen und garantiert keinen geringeren Gesamtverbrauch

Notizen

Energie: weniger Instruction Fetch knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Weniger Befehle können den Front-End-Anteil senken. Loads, Register und Laufzeit bleiben. Weniger Instruktionen sind nicht automatisch weniger Joule. Ein Vektorbefehl → viele Elemente parallel. Skalar: dutzende Loads/Adds/Branches aus dem I-Cache. Verständnischeck: Garantieren weniger Befehle weniger Energie? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 41 / 73

Vector Load/Store

- Breite Speicherpfade zum D-Cache / SRAM
- `vle32.v`: zusammenhängender Block → Vektorregister
- Ähnlich Burst-Idee: DRAM-/Bus-Lokalität besser ausnutzen

Notizen

Vector Load/Store knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. vle32.v mit VL 4 lädt vier 32-Bit-Werte, also 16 Byte, nicht vier Byte. Der Befehl darf intern mehrere Takte brauchen. Eine Cachezeile ist keine garantierte DRAM-Transaktion. Breite Speicherpfade zum D-Cache / SRAM. `vle32.v`: zusammenhängender Block → Vektorregister. Verständnischeck: Lädt VL 4 vier Bytes? Nein, 16 Nutzbytes. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 42 / 73

Chaining

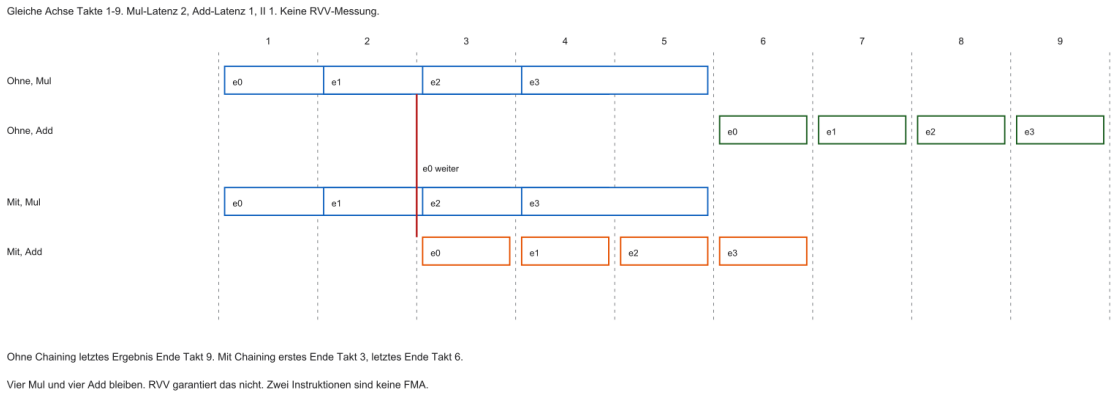


Abbildung 7: Gleiche Achse 1–9. Ohne Chaining letztes Ergebnis Ende Takt 9, mit Chaining Ende Takt 6.

- MAC: Mul dann Add — Zwischenergebnis sonst zurück ins Registerfile
- Chaining ist eine Option des Kerns. Eine FMA ist eine andere Instruktion mit einer Rundung
- Weniger Registerverkehr, höherer Durchsatz

Notizen

Chaining knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Mul startet in den Takten 1 bis 4 und endet 2 bis 5. Ohne Chaining startet Add in 6 bis 9 und endet dort. Mit elementweisem Weiterreichen startet Add in 3 bis 6. Das erste Ergebnis ist Ende Takt 3 statt 6, das letzte Ende Takt 6 statt 9. Die Rechenzahl bleibt gleich. Das ist kein RVV-Zeitversprechen und keine FMA. MAC: Mul dann Add — Zwischenergebnis sonst zurück ins Registerfile. Chaining ist eine Option des Kerns. Eine FMA ist eine andere Instruktion mit einer Rundung. Verständnischeck: Was spart das Modell? Wartezeit, nicht Multiplikationen. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 43 / 73

RVV und TinyML

- TinyML: Inferenz auf sehr knappen Energie-/SRAM-Budgets
- RVV-fähige Kerne: flexibel (noch volle CPU) + Vektor-Durchsatz
- Typisch: Keyword Spotting, kleine Sensor-Modelle am Edge

Notizen

RVV und TinyML knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. TinyML ist Inferenz bei knappem Speicher und Energie. RVV beschleunigt nur, wenn der Kernel sie benutzt. Firmware kann ohne Betriebssystem laufen. Ein vorhandenes TFLM-RVV-Build liegt hier nicht vor. TinyML: Inferenz auf sehr knappen Energie-/SRAM-Budgets. RVV-fähige Kerne: flexibel (noch volle CPU) + Vektor-Durchsatz. Verständnischeck: Nutzt TFLM automatisch RVV? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 44 / 73

Grenzen von Vektoren

- Registerfile bleibt: lesen/schreiben kostet Energie und limitiert Skalierung
- Ein großes Modellfile beweist den Registerengpass nicht. Form und Reuse entscheiden
- Nächste Stufe: Daten fließen ohne ständiges Zurückschreiben — Systolic Arrays

Notizen

Grenzen von Vektoren knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Registerports kosten Energie. Ein großes Modellfile beweist nicht, dass das Registerfile der Engpass ist. Ein systolisches Gitter ist eine andere Entscheidung, keine Pflichtstufe. Registerfile bleibt: lesen/schreiben kostet Energie und limitiert Skalierung. Sehr große Modelle: Pfad ALU → Register wird Engpass. Verständnischeck: Beweist die Dateigröße den Engpass? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 45 / 73

Kurzfazit RVV

- Die Software verarbeitet lange Arrays in Stücken der Länge VL
- VLEN ist die Registerbreite in Bit, VL die aktive Elementzahl
- Dieselbe Binärdatei braucht passende ISA, XLEN und ABI
- Weniger Befehlsstrom kann helfen. DLP ist die Datenparallelität, nicht der Befehl selbst
- Loads und Chaining können Wartezeit senken. Chaining ist keine RVV-Garantie

Notizen

Kurzfazit RVV knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Die Schleife benutzt das zurückgegebene VL, nicht eine feste VLEN. Loads heben die Bandbreite nicht auf. Chaining ist optional. Physische Parallelität folgt nicht aus der Bitbreite. Ein Befehl, lange Arrays. VLEN/VL abstrahiert Hardwarebreite. Verständnischeck: Welche Größe rückt den Zeiger? VL mal 4 Byte. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 46 / 73

Aufgabe — AVL 40

VLMAX ist 32. Nach 30 Schritten mit VL 32 bleiben 40 Elemente. Welche letzten VL-Werte sind zulässig, und wie viele Iterationen ergeben 1000 Elemente?

Notizen

Aufgabe — AVL 40 knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. AVL 40 und VLMAX 32 liegen zwischen 32 und 64. Die Untergrenze ist aufgerundet 20. VL 16 wäre zu klein. Danach ist der Rest exakt die nächste VL. 20 plus 20 sind je 80 Byte, 32 plus 8 sind 128 dann 32 Byte. Verständnischeck: Wäre 16 plus 24 zulässig? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 47 / 73

Was ist eine NPU?

- Neural Processing Unit: Beschleuniger für Inferenz (manchmal Training)
- Namen: TPU (Google), Neural Engine (Apple), NVDLA, …
- Nicht jede NPU ist ein systolisches Gitter oder nur für Matrizen. Kontrolle bleibt im System
- CPU füttert Adressen/Puffer (ähnlich DMA-Aufträgen)

Notizen

Was ist eine NPU knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. NPU ist eine Beschleunigerklasse, kein einheitlicher Befehlssatz. Nicht jede NPU ist ein systolisches Gitter oder nur Matrizen. DMA und Aufträge sind eine mögliche Organisation. Neural Processing Unit: Beschleuniger für Inferenz (manchmal Training). Namen: TPU (Google), Neural Engine (Apple), NVDLA, …. Verständnis-check: Ist NPU ein fester Befehlssatz? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 48 / 73

NPU vs. GPU vs. CPU

	Optimierung	Stärke	Schwäche
CPU	Latenz, Kontrollfluss	flexibel, auch kleine Modelle	braucht passende Datenversorgung
GPU	Durchsatz, viele Threads	programmierbar, breite Lasten	Energie hängt von Modell und Batch ab
NPU	unterstützte neuronale Operatoren	viele lokale MACs bei passender Form	unpassende Operatoren bleiben auf der CPU

Kombination ist möglich. Milliwatt gegen Watt ohne Messung bleibt offen.

Notizen

NPU vs. GPU vs. CPU knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Eine CPU kann für ein kleines Modell passen. Eine GPU ist nicht automatisch am Edge unbrauchbar. Milliwatt gegen Watt ohne Messung streichen. Latenz und Durchsatz sind verschieden. Verständnischeck: Kann die CPU reichen? Ja. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 49 / 73

Flaschenhals: Register / Verdrahtung

- Wunsch: zehntausende MACs/Takt
- Jede MAC braucht Operanden — unmöglich, unendlich viele Ports zum SRAM
- Routing und Multiplexer begrenzen die Breite
- Lösung: Daten lokal zwischen Zellen weiterreichen

Notizen

Flaschenhals: Register / Verdrahtung knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. 65536 MACs mal zwei Int8-Operanden wären ohne Reuse 131072 Byte je Takt. Das ist ein hypothetischer Bedarf, keine TPU-Messung. Mehr Multiplizierer brauchen Datenwege. Wunsch: zehntausende MACs/Takt. Jede MAC braucht Operanden — unmöglich, unendlich viele Ports zum SRAM. Verständnischeck: Warum nicht nur mehr Multiplizierer? Die Operanden müssen ankommen. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 50 / 73

Systolic Arrays

- 2D-Gitter aus MAC-Zellen
- Nur der Rand spricht mit dem Speicher
- Ergebnisse/Operanden wandern zu Nachbarn — ohne Systembus pro Schritt

Notizen

Systolic Arrays knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Im Lehrgitter wandert A nach rechts und B nach unten, C bleibt in der Zelle. Der Rand ist ein vereinfachtes Modell. Nachbarverbindungen ersetzen Vorladen und Readout nicht. 2D-Gitter aus MAC-Zellen. Nur der Rand spricht mit dem Speicher. Verständnischeck: Verschwindet der Speicher? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 51 / 73

„Systolisch“ — Namensbild

- Systole: rhythmische Pumpwelle
- Takt pumpt Werte wellenartig durch das Gitter
- Pro Zyklus: Daten eine Zelle weiter (rechts/unten je nach Dataflow)

Notizen

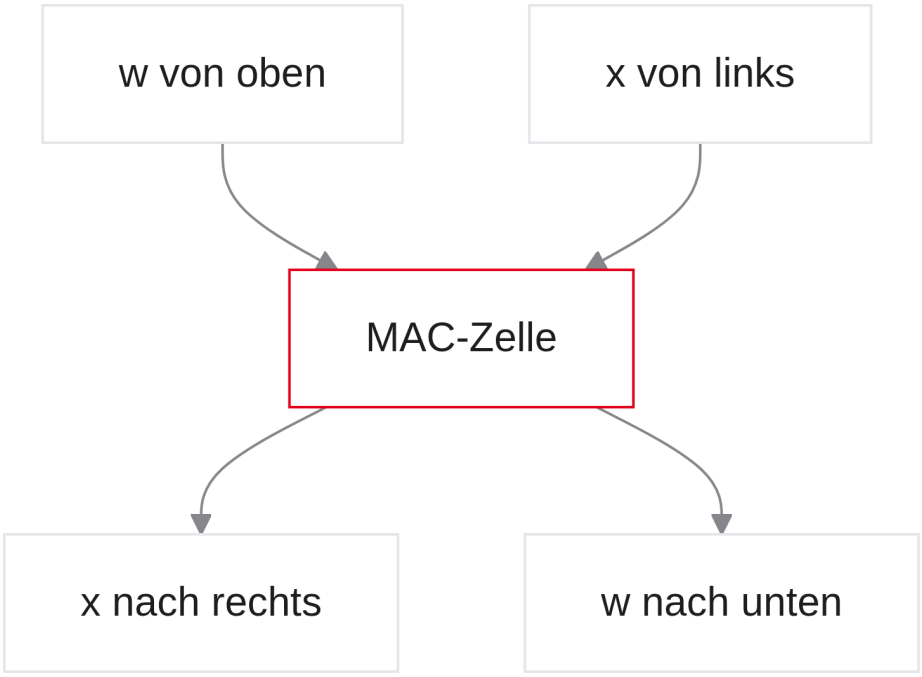
„Systolisch“ — Namensbild knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Ein Takt schiebt im Modell eine Zelle weiter. Leere Plätze rechnen nicht. Gleiche k-Indizes müssen in derselben Zelle zusammentreffen. Systole: rhythmische Pumpwelle. Takt pumpt Werte wellenartig durch das Gitter. Verständnischeck: Warum nicht alles ungeordnet einspeisen? Die Paare müssen sich treffen. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 52 / 73

Eine systolische Zelle

Pro Takt typisch:

1. Empfängt x (links) und w (oben)
2. MAC auf interne Teilsumme
3. Reicht x nach rechts und w nach unten weiter



Notizen

Eine systolische Zelle knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. C startet bei null. Ein gültiges Paar addiert A mal B. A wird original weitergereicht, nicht als Produkt. Für C00 erst 1 mal 5 gleich 5, dann plus 2 mal 7 gleich 19. Verständnischeck: Warum A weiterreichen? Die Nachbarzelle braucht dasselbe A. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 53 / 73

Weniger RAM-Zugriffe

- Ein Wert wird einmal eingespeist und wandert durch viele Zellen
- Viele MACs ohne erneuten Cache-/DRAM-Load
- Im Lehrfall steigt die Intensity von 0,25 auf 0,5 Op/Byte. Der Knick liegt bei 1

Notizen

Weniger RAM-Zugriffe knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. 16 Ops auf 64 Byte sind 0,25 Op je Byte, auf 32 Byte 0,5. Der Knick liegt bei 1. Beide Punkte bleiben speicherbegrenzt. Lokale Bewegung kostet ebenfalls Energie. Ein Wert wird einmal eingespeist und wandert durch viele Zellen. Viele MACs ohne erneuten Cache-/DRAM-Load. Verständnischeck: Beseitigt die doppelte Intensity die Grenze? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 54 / 73

Datenfluss füllt das Array

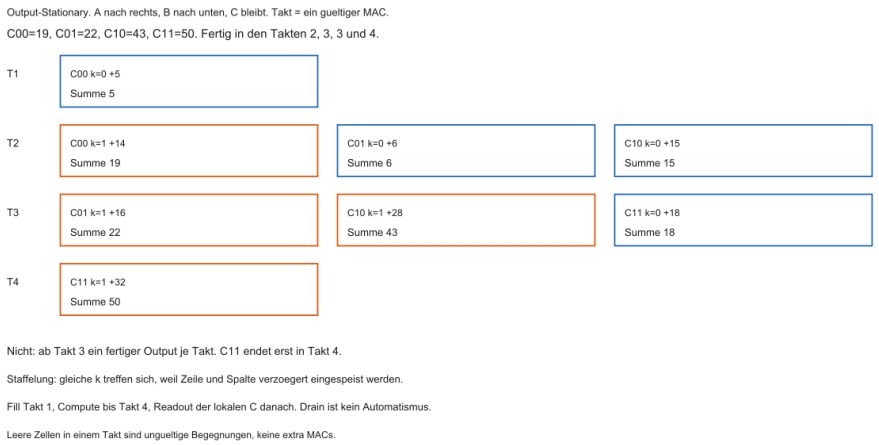


Abbildung 8: C00 fertig in Takt 2, C01 und C10 in Takt 3, C11 in Takt 4.

2×2-Idee (schematisch):

- Takt 1: erste Paare treffen sich, Summen starten
- Takt 2: Wellen wandern, neue Werte am Rand
- Im 2×2-Output-Stationary-Beispiel enden die vier Summen in den Takten 2, 3, 3 und 4
- Das ist kein Ergebnis je Takt ab Takt 3. Readout der lokalen Summen kommt danach

Notizen

Datenfluss füllt das Array knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. A kommt in Zeile i zur Zeit 1 plus k plus i, B in Spalte j zur Zeit 1 plus k plus j. C00 ist in Takt 1 gleich 5 und in Takt 2 fertig mit 19. C11 wird in Takt 3 gleich 18 und in Takt 4 gleich 50. Acht gültige MACs, nicht ein Ergebnis je Takt ab Takt 3. Takt 1: erste Paare treffen sich, Summen starten. Takt 2: Wellen wandern, neue Werte am Rand. Verständnischeck: Warum startet C11 später? Zeile und Spalte sind gestaffelt. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 55 / 73

Weight-Stationary



Abbildung 9: Output-Stationary hält C. Weight-Stationary hält das Gewicht und muss es trotzdem laden.

- Gewichte während einer Inferenz oft konstant
- Variante: Weights in Zellen „parken“, nur Activations durchschieben
- Weniger Bewegungen der großen Konstanten-Matrizen

Notizen

Weight-Stationary knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Weight-Stationary hält Gewichte nur für den benannten Zeitraum. Ein Tilewechsel kann neu laden. Aktivierungen und Teilsummen bewegen sich weiter. Gewichte während einer Inferenz oft konstant. Variante: Weights in Zellen „parken“, nur Activations durchschieben. Verständnischeck: Werden Gewichte nie wieder geladen? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 56 / 73

Beispiel: Google TPU v1

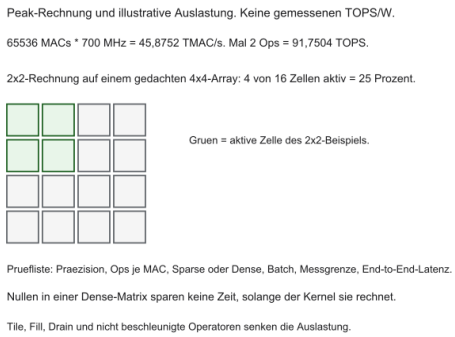


Abbildung 10: 91,75 TOPS Peak bei zwei Ops je MAC. Ein 2×2-Tile nutzt 4 von 16 gedachten Zellen.

- $256 \times 256 = 65\,536$ Int8-MACs. Bei 700 MHz sind das 45,8752 TMAC/s
- Mit zwei Ops je MAC: 91,7504 TOPS, in der Originalarbeit oft 92 TOPS. Das ist Peak, kein End-to-End
- TPU v1 ist ein Datacenter-Baustein, kein Milliwatt-Edge-Teil

Notizen

Beispiel: Google TPU v1 knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. 256 mal 256 ist 65536. Mal 700 MHz sind 45,8752 Billionen MACs je Sekunde. Mal zwei Ops sind 91,75 TOPS, gerundet 92. Das ist Peak, keine Inferenzrate. Ein 2 mal 2 Tile in 4 mal 4 belegt räumlich 25 Prozent. Über vier Takte sind zeitlich 8 durch 64 gleich 12,5 Prozent. Auf dem 2 mal 2 selbst sind es 50 Prozent. $256 \times 256 = 65\,536$ Int8-MACs. Bei 700 MHz sind das 45,8752 TMAC/s. Mit zwei Ops je MAC: 91,7504 TOPS, in der Originalarbeit oft 92 TOPS. Das ist Peak, kein End-to-End. Verständnischeck: Was sind 92 TOPS? Ein gerundeter Rechenpeak, kein End-to-End-Wert. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 57 / 73

NPUs am Edge

- SoCs mit kleineren Arrays (z. B. 32×32 / 64×64)
- CPU, Vorverarbeitung und NPU gehören zusammen. Ein Betriebssystem ist optional
- Die CPU schläft nur, wenn Versorgung, Wakeup und übrige Aufgaben das erlauben

Notizen

NPUs am Edge knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. 32 mal 32 ist

eine Schemazeichnung, keine SoC-Konfiguration. Die CPU schläft nur, wenn Versorgung, Steuerung und übrige Aufgaben das erlauben. SoCs mit kleineren Arrays (z. B. 32×32 / 64×64). CPU: OS, Netz; NPU: Bild/Audio-Inferenz (oft per DMA gefüttert). Verständnischeck: Schläft die CPU bei jeder Inferenz? Nicht zwingend. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 58 / 73

Grenzen systolischer Arrays

- Starr an Array-/Tile-Größen — Padding bei ungünstigen Layern
- Dense-Matrizen stark; Sparse (viele Nullen) weniger effizient
- Kein General Purpose — kein TCP/IP-Stack auf der NPU

Notizen

Grenzen systolischer Arrays knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. 25 Prozent belegte Zellen sind nicht 25 Prozent zeitliche Auslastung. Dichte Nullen sparen nur mit passender Sparse-Ausführung MACs. Nicht jede NPU ist gleich starr. Starr an Array-/Tile-Größen — Padding bei ungünstigen Layern. Dense-Matrizen stark; Sparse (viele Nullen) weniger effizient. Verständnischeck: Sind Belegung und Auslastung gleich? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 59 / 73

Kurzfazit NPU

- Systolic Arrays: lokale Weitergabe statt Von-Neumann-Pendeln
- Wellenartiger Datenfluss, hohe Wiederverwendung
- Weight-Stationary spart Bewegungen
- TOPS je Watt brauchen gemessene Leistung derselben Grenze, nicht nur den Peak

Notizen

Kurzfazit NPU knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Lokale Weitergabe senkt bestimmten Verkehr. TOPS je Watt brauchen gemessene Watt derselben Grenze. 92 TOPS ohne Leistung sind keine TOPS je Watt. Systolic Arrays: lokale Weitergabe statt Von-Neumann-Pendeln. Wellenartiger Datenfluss, hohe Wiederverwendung. Verständnischeck: Kann man TOPS je Watt ohne Watt rechnen? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 60 / 73

Aufgabe — Takt 4

Wann ist C11 im 2×2-Output-Stationary-Beispiel fertig? Ist 92 TOPS eine End-to-End-Leistung?

Notizen

Aufgabe — Takt 4 knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. C11 erhält 3 mal 6 gleich 18 in Takt 3 und 4 mal 8 gleich 32 in Takt 4, Summe 50. Der Readout ist ein eigener Schritt. Die TOPS-Rechnung ist 65536 mal 700 Millionen mal 2. Verständnischeck: Ist C11 dann schon außen gelesen? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 61 / 73

Edge-AI-Software: das Problem

- Training oft in Python (TensorFlow/PyTorch) auf GPUs — große Float32-Modelle
- MCU: kein Python, knappes SRAM, oft keine FPU
- Wie wird das Cloud-Modell zu Bare-Metal-C?

Notizen

Edge-AI-Software: das Problem knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Das Modellarray ist Daten. Kernel in der Firmware führen Operatoren aus. Cloud ist nicht unbegrenzt. Nicht jede MCU hat eine FPU. Training oft in Python (TensorFlow/PyTorch) auf GPUs — große Float32-Modelle. MCU: kein Python, knappes SRAM, oft keine FPU. Verständnischeck: Wird aus dem Array automatisch ein C-Programm? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 62 / 73

TensorFlow Lite for Microcontrollers (TFLM)

- Kleine C++-Runtime. Ohne verpflichtendes OS und mit Arena möglich. Der übrige Stack ist nicht automatisch heapfrei
- Nimmt kompaktes Modell, interpretiert Layer (Conv2D, Dense, …)
- Rechnet mit Firmware-Kerneln. Eine NPU braucht ein passendes Backend, nicht nur das Silizium

Notizen

TensorFlow Lite for Microcontrollers (TFLM) knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. TFLM ist eine kleine C++-Runtime. Sie kann ohne Betriebssystem mit einer Arena arbeiten. Das ganze Firmwareprojekt ist dadurch nicht heapfrei. Ein NPU-Backend ist extra. Hier liegt kein Commit vor. Kleine C++-Runtime für MCUs (ohne OS, ohne `malloc`). Nimmt kompaktes Modell, interpretiert Layer (Conv2D, Dense, …). Verständnischeck: Wer rechnet: Bytes oder Kernel? Die Kernel. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 63 / 73

Workflow Embedded AI

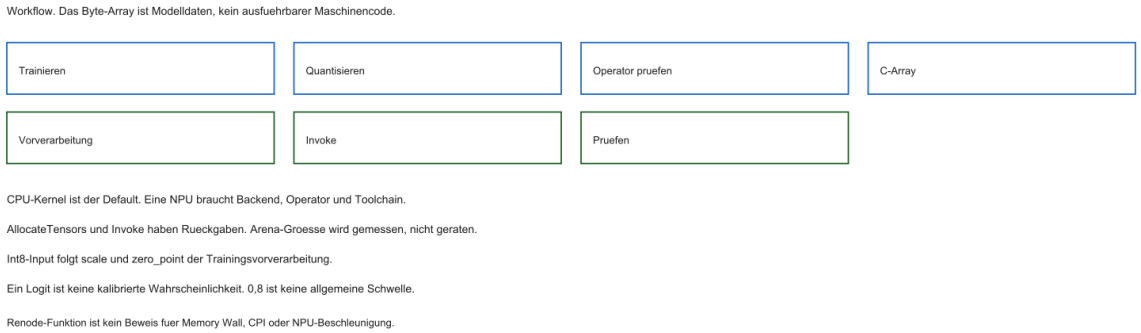


Abbildung 11: Das C-Array sind Modelldaten. Invoke läuft über Kernel, nicht als Modell-Maschinencode.

1. Trainieren (Cloud / Python)
2. Konvertieren → `.tflite`
3. Export als C-Byte-Array (`xxd` o. Ä.)
4. Linken in Flash (`.rodata`)
5. Firmware: `Invoke()` / Interpreter

Notizen

Workflow Embedded AI knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Offline entstehen geprüfte Bytes, online laufen Kernel. .rodata ist ein Linkerbereich, nicht automatisch Flash. Der Export allein genügt nicht. Verständnischeck: Warum genügt das Array nicht? Operatoren, Laufzeit und Speicher fehlen. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 64 / 73

Quantisierung

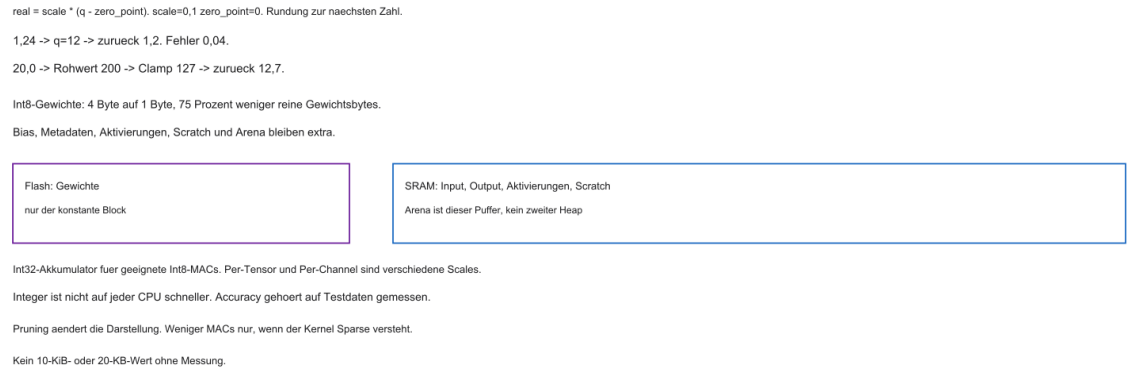


Abbildung 12: 1,24 wird 12 und zurück 1,2. 20 wird auf 127 begrenzt. Flash und SRAM bleiben getrennt.

- Float32: 4 Byte/Gewicht, teuer ohne FPU
- Reine Float32-Gewichte werden als Int8 um 75 Prozent kleiner. Bias, Aktivierungen und Arena nicht automatisch
- Integer ist nicht auf jeder CPU schneller. Den Accuracy-Verlust messen, nicht annehmen

Notizen

Quantisierung knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. real gleich s mal (q minus z). Bei s 0,1 und z 0 wird 1,24 zu 12 und zurück 1,2, Fehler 0,04. 20 wird auf 127 geklemmt, Rückwert 12,7, Fehler 7,3. Minus 20 wird minus 128, Rückwert minus 12,8. Bei z minus 3 ist die kodierte Null minus 3. 75 Prozent gelten nur für reine Gewichtbytes von 4 auf 1. Float32: 4 Byte/Gewicht, teuer ohne FPU. Reine Float32-Gewichte werden als Int8 um 75 Prozent kleiner. Bias, Aktivierungen und Arena nicht automatisch. Verständnischeck: Warum ist die Null bei z minus 3 gleich minus 3? Weil s mal null bleibt null. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 65 / 73

Pruning

- Viele Gewichte nahe null nach dem Training
- Pruning braucht eine sparsityfähige Darstellung und einen passenden Kernel
- Flash-Ersparnis und weniger ausgeführte MACs sind zwei verschiedene Folgen

Notizen

Pruning knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Vier dichte Bytes bleiben vier Bytes, auch wenn zwei null sind. Sparse-Speicher braucht Indizes. Ein Nullgewicht spart nur dann eine MAC, wenn der Kernel sie überspringt. Viele Gewichte nahe null nach dem Training. Pruning braucht eine sparsityfähige Darstellung und einen passenden Kernel. Verständnischeck: Spart eine Null automatisch eine MAC? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 66 / 73

Geplante Aktivitätserkennung

Geplanter Anwendungsfall, kein vorhandenes Labor:

- Unter 20 KB ist ein hypothetisches Modellziel, kein vorhandenes Modell
- Ein 3-Achsen-Fenster wäre später an die I2C-Pipeline anzuschließen
- Klassen Stehen, Gehen, Laufen sind noch kein Schrittzähler
- Renode, `.repl` und Runtime liegen hier nicht vor

Notizen

Geplante Aktivitätserkennung knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Stehen, Gehen und Laufen sind Fensterklassen, keine gezählten Schritte. Unter 20 KB ist ein hypothetisches Ziel, kein vorhandenes Modell. Es gibt keine .repl. Kleines quantisiertes Modell (< 20 KB). Eingabe: 3-Achsen-Beschleunigung (Anknüpfung I2C, Block 5). Verständnischeck: Ist jedes Gehen-Fenster ein Schritt? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 67 / 73

Modell als C-Array

```
/* Platzhalter, kein gültiges Modell und keine Lizenzdatei. */
const unsigned char step_model_tflite[]
    __attribute__((aligned(4))) = {
    0x1c, 0x00, 0x00, 0x00, 0x54, 0x46, 0x4c, 0x33,
    /* … weitere Bytes … */
};
```

Notizen

Modell als C-Array knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Die acht Bytes mit TFL3 sind kein gültiges Modell und kein Maschinencode. const legt nicht allein ins Flash. Eine Ausrichtung ist compilerspezifisch. Verständnischeck: Beweist TFL3 ein Netz? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 68 / 73

Tensor Arena

```
/* Pseudocode. Kein TFLM-Build und kein gültiges Modell.
   Ein Kontext bleibt über Setup und Run gültig. */
static int ready;

int setup(void) {
    if (model_blob_invalid()) return -1;
    if (plan_tensors() != 0) return -3;
    ready = 1;
    return 0;
}
```

- 1024 Byte Arena sind eine Lehrannahme
- Der lokale Interpreter der alten Fassung endete mit der Funktion

Notizen

Tensor Arena knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Der alte lokale Interpreter endete mit return. Setup und Run brauchen denselben langlebigen Kontext. 1024 Byte sind eine Lehrannahme, kein Budget für ein 20-KB-Modell. Ohne gültigen Blob wird nicht dereferenziert. Ein zu kleiner Puffer lässt die Planung scheitern. 1024 Byte Arena sind eine Lehrannahme. Der lokale Interpreter

der alten Fassung endete mit der Funktion. Verständnischeck: Warum genügt return 0 nicht? Das Objekt und das Budget bleiben sonst offen. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 69 / 73

Inferenz-Zyklus

```
/* Pseudocode. src hat m Float32-Werte, das Ziel m Int8-Elemente
int run_window(const float *src, size_t m) {
    if (!ready || src == 0) return -1;
    if (input_elements() != m) return -6;
    for (size_t i = 0; i < m; i++)
        input_i8[i] = quantize_f32(src[i]);
    if (invoke() != 0) return -8;
    return status_only(); /* Klasse und Score getrennt */
}
```

- `sizeof` eines Zeigers zählt nicht die Fensterelemente
- Drei Ausgabebytes beweisen keine drei Klassen und keine Schrittzahl

Notizen

Inferenz-Zyklus knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. m Float32-Werte brauchen 4m Quellbytes und m Int8-Ziele. sizeof eines Zeigers zählt den Puffer nicht. Invoke ohne Erfolg liefert keinen Score. Drei Bytes beweisen keine Klasse Gehen. Status, Index und Score bleiben getrennt. Zwölf Floatwerte wären 48 Quellbytes und 12 Int8-Ziele. `sizeof` eines Zeigers zählt nicht die Fensterelemente. Drei Ausgabebytes beweisen keine drei Klassen und keine Schrittzahl. Verständnischeck: Brauchen 12 Floatwerte 12 Bytes? Nein, 48. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 70 / 73

Renode: virtuelle Stimuli

- Geplanter späterer Ausblick. Keine .repl und kein Modell liegen vor
- Eine Konsole wäre eine funktionale Beobachtung, keine Zyklusmessung

Notizen

Renode: virtuelle Stimuli knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Ohne Plattform, Firmware und Modell gibt es keine Konsole und kein .resc. Eine spätere Klasse wäre funktional, keine Energiezahl. Geplanter späterer Ausblick. Keine .repl und kein Modell liegen vor. Eine Konsole wäre

eine funktionale Beobachtung, keine Zyklusmessung. Verständnischeck: Beweist die Konsole Zyklen? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 71 / 73

Erwartung: CPU ohne NPU

- RV32IM ohne FPU kann Float in Software rechnen. Der Aufwand ist hier nicht gemessen
- Eine funktionale Simulation belegt weder die Memory Wall noch eine NPU-Beschleunigung
- Int8 auf der CPU kann sinnvoll sein. Eine CPU ohne NPU ist nicht automatisch falsch

Notizen

Erwartung: CPU ohne NPU knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. RV32IM ohne FPU kann Float in Software rechnen. Der Aufwand ist nicht gemessen. Int8 auf der CPU kann passen. MIPS sind keine Megahertz. RV32IM ohne FPU kann Float in Software rechnen. Der Aufwand ist hier nicht gemessen. Eine funktionale Simulation belegt weder die Memory Wall noch eine NPU-Beschleunigung. Verständnischeck: Ist die CPU ohne NPU falsch? Nein. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 72 / 73

Zusammenfassung Block 8

- DLP ist Datenparallelität. SIMD und RVV sind Umsetzungen, keine Definition
- Systolische Arrays reduzieren bestimmten globalen Verkehr und brauchen Fill und Readout
- TFLM-Bytes sind Modelldaten. Die Kernel führen die Operatoren aus
- Quantisierung spart bestimmte Gewichtbytes, keine garantierte Genauigkeit

Notizen

Zusammenfassung Block 8 knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Die Ansätze eint passende Datenversorgung und lokale Wiederverwendung. DLP, RVV und systolische Arrays bleiben verschiedene Mechanismen. DLP ist Datenparallelität. SIMD und RVV sind Umsetzungen, keine Definition. Systolische Arrays reduzieren bestimmten globalen Verkehr und brauchen Fill und Readout. Verständnischeck: Welche Aussage verbindet sie? Gleichartige Rechnung braucht passende Versorgung. Die nächste Folie nimmt die noch offene Frage auf. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.

Folie 73 / 73

Ausblick und Abschluss

- Geplante Aktivitätserkennung, hier nicht ausgeführt
- Es fehlen Modell, Tensorvertrag, Kernel und Plattform

Vielen Dank.

Notizen

Ausblick und Abschluss knüpft an die bisherige Frage nach Rechnung und Datenbewegung an. Geprüft sind Rechnungen und Semantik. Nicht geprüft sind Kompilierung, Inferenz und Renode. Eine spätere Integration braucht Modell, Vertrag, Kernel und Plattform. Geplante Aktivitätserkennung, hier nicht ausgeführt. Es fehlen Modell, Tensorvertrag, Kernel und Plattform. Verständnischeck: Was wurde nicht ausgeführt? Firmware, Training und Simulation. Ein späterer Versuch braucht Modell, Kernel und Plattform. Zahlen sind Lehrannahmen. Code ist Lehrcode und wurde nicht kompiliert, trainiert oder ausgeführt.